

COMPUTACIÓN PARALELA Y DISTRIBUIDA

Laboratorio 1 — ¿Más procesadores significa mayor velocidad?

Semana 1

1. Propósito

Realizar un primer experimento de computación paralela comparando una implementación secuencial con una implementación paralela de un problema sencillo. Se medirá el rendimiento y se analizará si aumentar el número de threads produce una aceleración proporcional.

2. Objetivos

- Implementar una solución secuencial para un problema de procesamiento de datos.
- Identificar las operaciones que pueden ejecutarse de manera independiente.
- Implementar una primera versión paralela utilizando OpenMP.
- Medir el tiempo de ejecución para diferentes cantidades de threads.
- Calcular speedup y eficiencia.
- Analizar experimentalmente el comportamiento del programa paralelo.

3. Problema

Se dispone de un vector de números reales de gran tamaño. Se desea calcular la suma de todos sus elementos.

$$S = \sum A[i], \quad i = 0, \dots, N-1$$

El programa deberá generar o inicializar el vector y calcular su suma. La solución debe producir un resultado correcto y reproducible.

4. Parte A — Implementación secuencial

- Genere un vector de números de tipo double.
- Implemente una versión secuencial del cálculo de la suma.
- Utilice un mecanismo de medición de tiempo apropiado, por ejemplo `std::chrono`.
- Realice varias ejecuciones para obtener mediciones representativas.

Se recomienda comenzar con $N = 10^8$ elementos y ajustar el tamaño según las características del equipo.

5. Parte B — Identificación del paralelismo

Antes de implementar la versión paralela, analice el ciclo que recorre el vector.

- ¿Qué operaciones pueden realizarse independientemente?
- ¿Qué información debe combinarse al finalizar el procesamiento?
- ¿Qué problema aparecería si varios threads modificaran directamente una misma variable de suma?

6. Parte C — Implementación con OpenMP

Implemente una versión paralela utilizando OpenMP. Distribuya las iteraciones del ciclo entre los threads y utilice un mecanismo apropiado para obtener la suma final.

En esta primera práctica no es necesario implementar manualmente mecanismos de sincronización. El objetivo es observar el efecto del paralelismo y medirlo.

7. Parte D — Experimento de rendimiento

Ejecute el programa utilizando diferentes cantidades de threads. Considere como mínimo los valores razonables para el equipo utilizado.

Threads	Tiempo (s)	Speedup	Eficiencia
1			
2			
4			
8			
16			
32			

Speedup: $S_p = T_1 / T_p$

Eficiencia: $E_p = S_p / p$

8. Parte E — Influencia del tamaño del problema

Repita el experimento para diferentes tamaños del vector. Se recomienda utilizar:

- $N = 10^6$
- $N = 10^7$
- $N = 10^8$

Compare cómo cambia el speedup cuando aumenta el tamaño del problema. Si el equipo no dispone de memoria suficiente, seleccione valores adecuados y registre la decisión.

9. Preguntas de análisis

1. ¿Qué parte del algoritmo puede ejecutarse en paralelo? Explique.
2. ¿Qué problema aparece al calcular la suma utilizando múltiples threads?
3. ¿El speedup obtenido es lineal? Justifique utilizando los resultados experimentales.

4. ¿A partir de qué cantidad de threads el rendimiento deja de mejorar significativamente?
5. ¿Qué ocurre cuando se utilizan más threads que núcleos disponibles?
6. ¿El tamaño del problema influye en el speedup obtenido? Explique.
7. Proponga al menos dos razones que podrían explicar por qué el speedup no aumenta proporcionalmente al número de threads.

10. Entregable

Entregue un informe breve acompañado del código fuente. El informe deberá contener:

- Descripción del problema y estrategia de paralelización.
- Código de las versiones secuencial y paralela.
- Tabla de tiempos, speedup y eficiencia.
- Gráfica de speedup en función del número de threads.
- Resultados para diferentes tamaños del problema.
- Respuestas a las preguntas de análisis.
- Conclusión: ¿más threads significan necesariamente mayor velocidad?

11. Consideraciones

- Utilice el mismo equipo y condiciones de ejecución para realizar las comparaciones.
- Realice varias ejecuciones y evite utilizar una única medición.
- Verifique que las versiones secuencial y paralela produzcan resultados equivalentes.
- Registre procesador, número de núcleos y memoria RAM del equipo utilizado.

12. Idea central

**Paralelizar no significa simplemente utilizar más threads.
Debemos medir, comparar y explicar el comportamiento obtenido.**